Day 114 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 3 reference

Watch: Naive Bayes, Clearly Explained!!! (StatQuest) | Understanding Multinomial and Bernoulli Naive Bayes (AI Wizardry)

`day114_naive_bayes.py`, segment by segment, in the order `main()` uses things. Naive Bayes is new; Day 98's tokeniser, logistic regression and calibration are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: c = class · w = word · α = smoothing · V = vocabulary size · $p_{w,c}$ = P(word present | class)

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
W15 = os.path.join(HERE, "..", "week-15-practical-nlp-pretrained")
sys.path.insert(0, W15)
with contextlib.redirect_stdout(io.StringIO()), contextlib.redirect_stderr(io.StringIO()):
    import day98_pretrained_transformers as d98
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
SIZES = [100, 300, 1000, 3000, 10000, 30000, 67349]
SEEDS = 5
ALPHAS = [0.01, 0.03, 0.1, 0.3, 1, 3, 10]
NEGATION = {"not", "no", "never", "nor", "nothing"}
```

- `with contextlib.redirect_stdout(...), contextlib.redirect_stderr(...):` — two context managers in one with statement: both outputs are silenced.
- `sys.stdout.reconfigure(encoding="utf-8")` — real text may contain characters Windows' default encoding cannot write.
- `NEGATION = {"not", ...}` — a set: membership tests are fast.

Segment 1 — Naive Bayes

tokens and encode

CODE

```
def tokens(text, ngrams=1):
    w = d98.words(text)
    return w + [a + " " + b for a, b in zip(w, w[1:])] if ngrams == 2 else w
class NaiveBayes:
    def __init__(self, kind="multinomial", alpha=1.0, ngrams=1):
        self.kind, self.alpha, self.ngrams = kind, alpha, ngrams
    def encode(self, texts, grow=False):
        ids, rows = [], []
        for r, t in enumerate(texts):
            toks = tokens(t, self.ngrams)
            for w in (toks if self.kind == "multinomial" else set(toks)):
                if grow and w not in self.vocab:
                    self.vocab[w] = len(self.vocab)
                if w in self.vocab:
                    ids.append(self.vocab[w]); rows.append(r)
        return np.array(ids, dtype=np.int64), np.array(rows, dtype=np.int64)
```

- `[a + " " + b for a, b in zip(w, w[1:])]` — zip with the list shifted by one gives every adjacent pair.
- `toks if self.kind == "multinomial" else set(toks)` — a set removes repeats: binary and Bernoulli count a word once.
- `self.vocab[w] = len(self.vocab)` — a new word gets the next free column number.

fit: training is counting

MATH

$$P(w \mid c) = (N_{c,w} + \alpha) / (N_c + \alpha V); \quad \text{Bernoulli: } p = (D_{c,w} + \alpha) / (n_c + 2\alpha)$$

CODE

```
def fit(self, texts, y):
    self.vocab = {}; ids, rows = self.encode(texts, grow=True); V = len(self.vocab); a =
        self.alpha
    self.prior = np.log(np.bincount(y, minlength=2) / len(y))
    N = np.stack([np.bincount(ids[y[rows] == c], minlength=V) for c in (0, 1)]).astype(float)
    if self.kind == "bernoulli":
        n = np.bincount(y, minlength=2)[:, None]
        p = (N + a) / (n + 2 * a)
        self.lp, self.lq = np.log(p), np.log(1 - p)
    else:
        self.lp = np.log((N + a) / (N.sum(1, keepdims=True) + a * V))
    return self
```

- `np.bincount(ids[y[rows] == c], minlength=V)` — how often each word occurs in class c's texts: one pass, no loop over words.
- `N.sum(1, keepdims=True)` — all word slots of each class, kept as a column so the division broadcasts.
- `np.stack([...])` — two rows, one per class.

log_odds: adding up the evidence

MATH

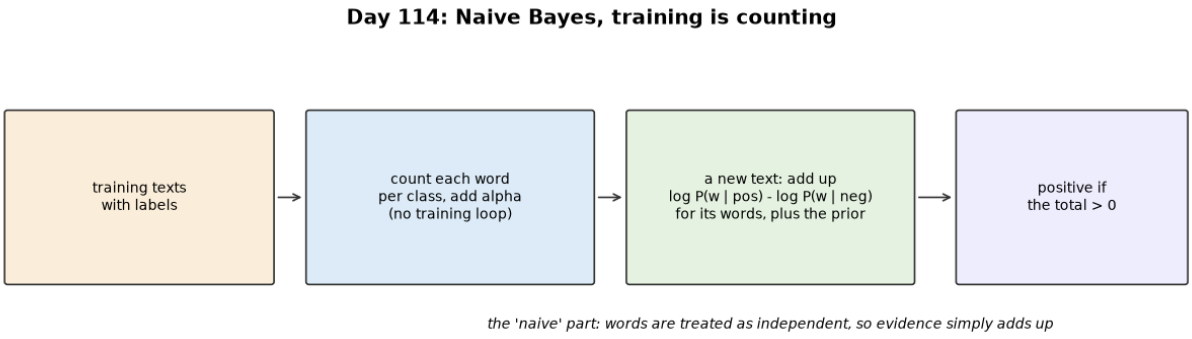
$$log\text{-}odds = log\ P(+)/P(-) + \Sigma_{w\ in\ text}\ [log\ P(w \mid +) - log\ P(w \mid -)]$$

CODE

```
def log_odds(self, texts):
    ids, rows = self.encode(texts); n = len(texts)
    d = self.lp[1] - self.lp[0] if self.kind != "bernoulli" else (self.lp[1] - self.lq[1]) -
        (self.lp[0] - self.lq[0])
    s = self.prior[1] - self.prior[0] + np.bincount(rows, weights=d[ids], minlength=n)
    if self.kind == "bernoulli":
        s += self.lq[1].sum() - self.lq[0].sum()
    return s
def proba(self, texts):
    return 1 / (1 + np.exp(-self.log_odds(texts)))
```

- `np.bincount(rows, weights=d[ids], minlength=n)` — adds each text's word ratios: the weights are summed per row.
- `self.lq[1].sum() - self.lq[0].sum()` — Bernoulli: every absent word counts too, so start from all-absent and correct for the present ones.

SYSTEM DIAGRAM



Training is counting.

Segment 2 — The experiments

Helpers: Day 98's model, accuracy, negation

CODE

```
def lr_proba(bow, texts):
    ids, rows = bow.encode(texts)
    return 1 / (1 + np.exp(-bow.scores(ids, rows, len(texts))))
def accuracy(p, y):
    return float(np.mean((p > 0.5).astype(int) == y))
def negated(text):
    return any(w in NEGATION or w.endswith("n't") for w in d98.words(text))
def day98_distilbert():
    s = open(os.path.join(W15, "day98_run_output.txt"), encoding="utf-8").read()
    return float(re.search(r"DistilBERT\s+30,522\s+([0-9.]+)", s).group(1))
```

- `bow.scores(ids, rows, len(texts))` — Day 98's BagOfWords, unchanged: its score is the log-odds.
- `any(... for w in d98.words(text))` — True as soon as one word is a negation.

- `re.search(r"DistilBERT\s+30,522\s+([0-9.]+)", s).group(1)` — reads Day 98's result from its own output file, so the number is never typed by hand.

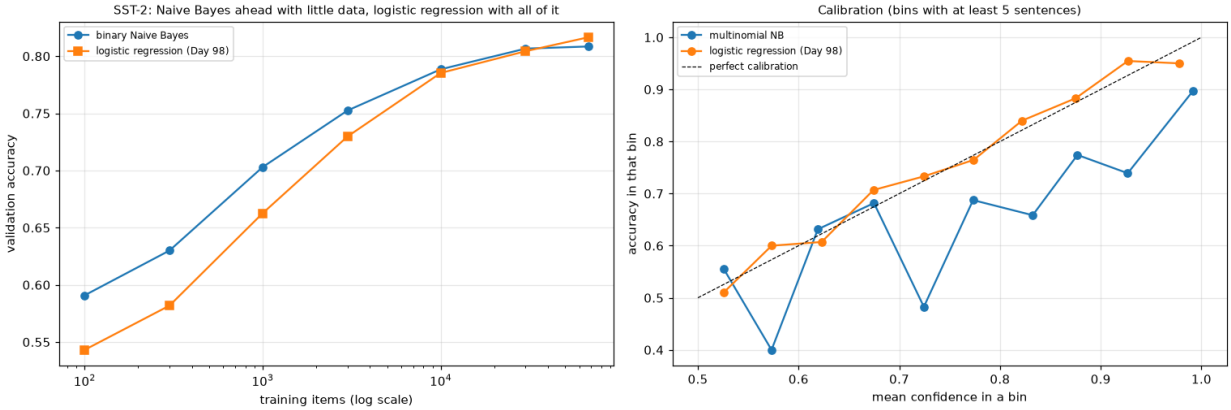
Doubling every sentence, inside demo_independence

CODE

```
double = [t + " " + t for t in Xva]
m = NaiveBayes("multinomial").fit(*train_data()); lo1, lo2 = m.log_odds(Xva), m.log_odds(double)
b = NaiveBayes("binary").fit(*train_data()); lb1, lb2 = b.log_odds(Xva), b.log_odds(double)
```

- `[t + " " + t for t in Xva]` — every sentence written twice: no new information.
- `NaiveBayes(...).fit(*train_data())` — * unpacks the (texts, labels) pair into two arguments.

REAL GRAPH



Real graph from the script: learning curves and calibration.

Quick Syntax Glossary — new or reinforced constructs

- `np.bincount(ids, weights=w, minlength=n)` — sum the weights per index.
- `zip(w, w[1:])` — adjacent pairs.
- `f(*pair)` — unpack a tuple into arguments.
- `with a(), b():` — two context managers at once.
- `re.search(pattern, s).group(1)` — the first bracketed part of a match.
- `{"a", "b"}` — a set literal.

End of Day 114 syntax reference. For the experiments and what they showed, see the Day 114 complete guide.